



كلية تكنولوجيا الصناعة والطاقة

Information Technology Department



جامعة برج العرب التكنولوجية

BORG AL ARAB TECHNOLOGICAL UNIVERSITY

INTRODUCTION TO C PROGRAMMING ESSENTIALS

First Year – Information Technology Program

spring 2026



LECTURE 7

C LANGUAGE

BASICS

ARRAYS



Dr. Dina Abdelhafize & Dr. Ghada Fathy



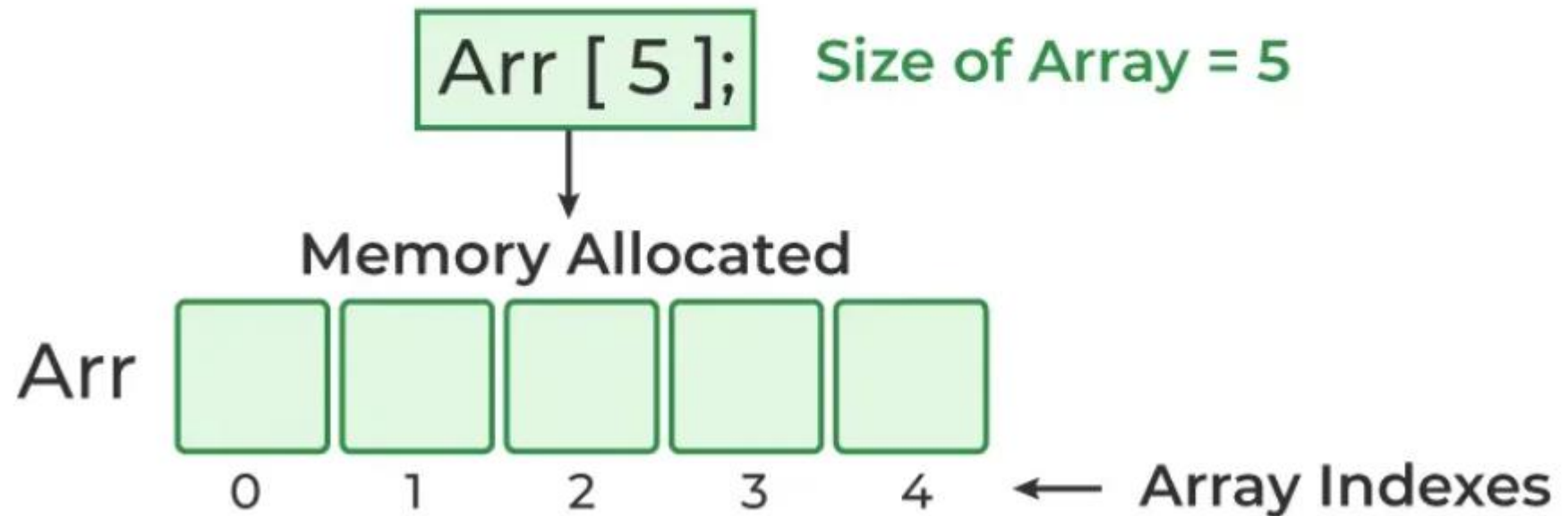
ARRAYS IN C

- Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.
- To create an array, define the data type (like `int`) and specify the name of the array followed by square brackets `[]`.
- To insert values to array, use a comma-separated list inside curly braces, and make sure all values are of the same data type
- `int myNumbers[] = {25, 50, 75, 100};`

ARRAY DECLARATION

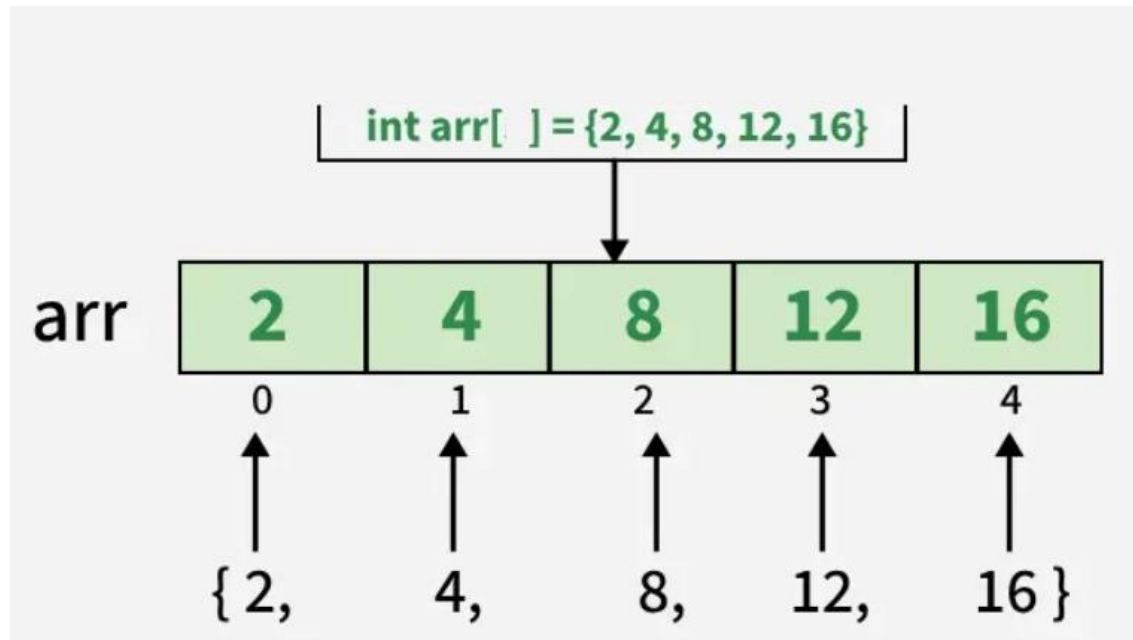
- When we declare an array in C, the compiler allocates the memory block of the specified size to the array name.

Array Declaration



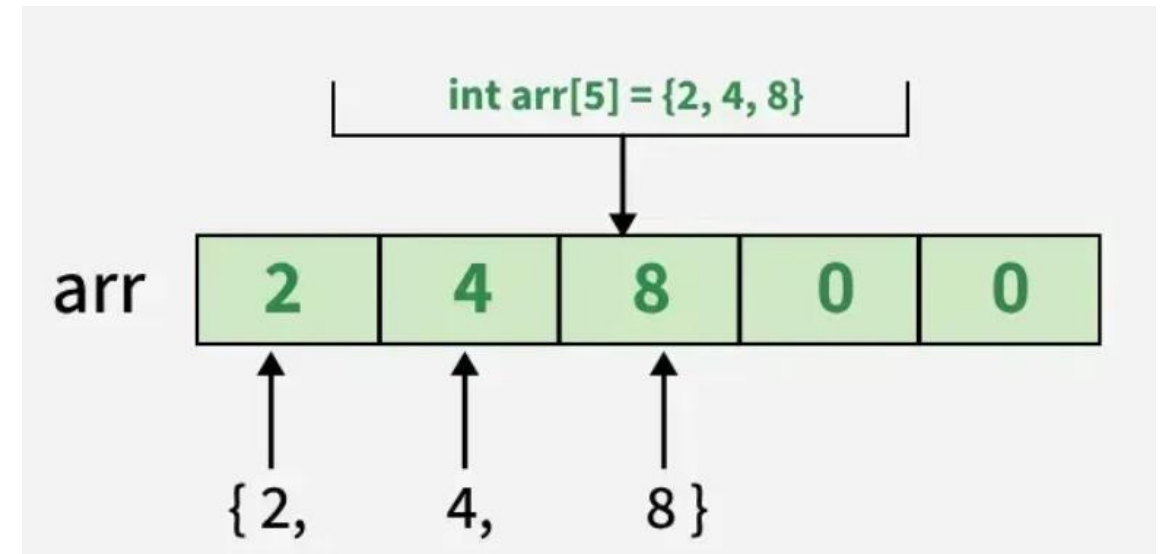
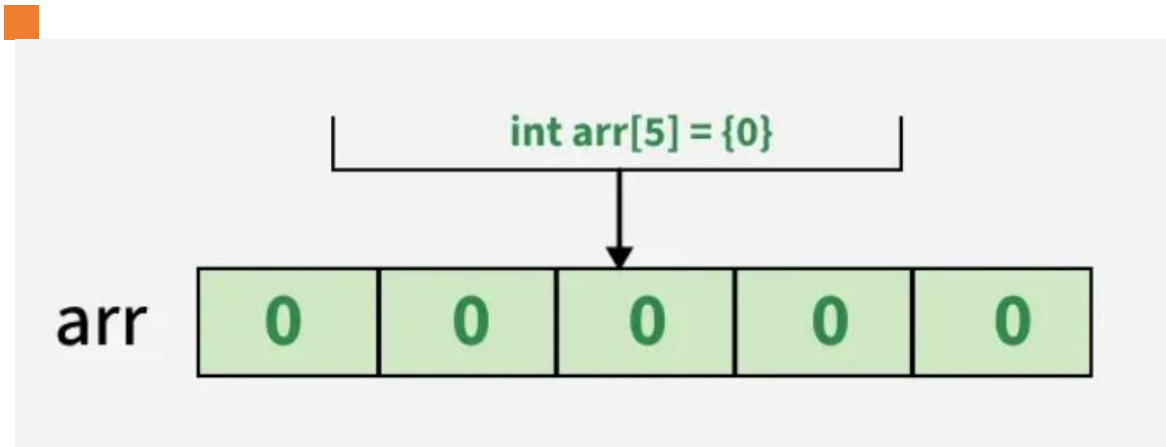
ARRAY INITIALIZATION

- When the array is declared or allocated memory, the elements of the array contain some garbage value. So, we need to initialize the array to some meaningful values.
- We can **skip** mentioning the size of the array if declaration and initialization are done at the same time.



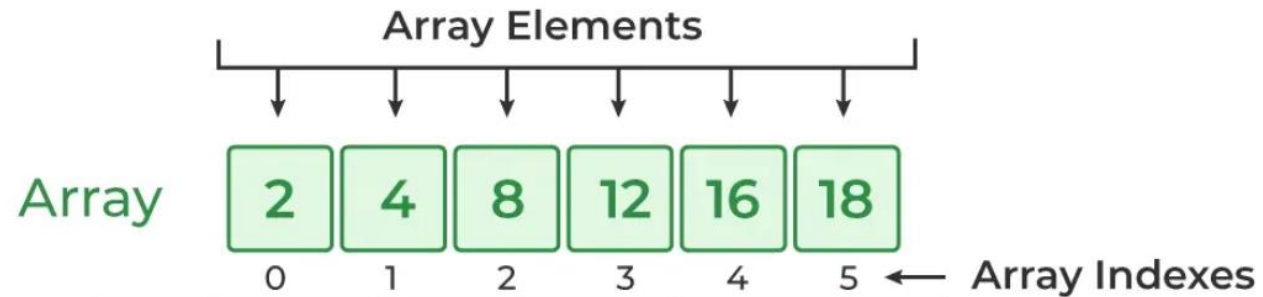
ARRAY INITIALIZATION

- We can also partially initialize while declaring. In this case, the remaining elements will be assigned the value 0 (or equivalent according to the type).

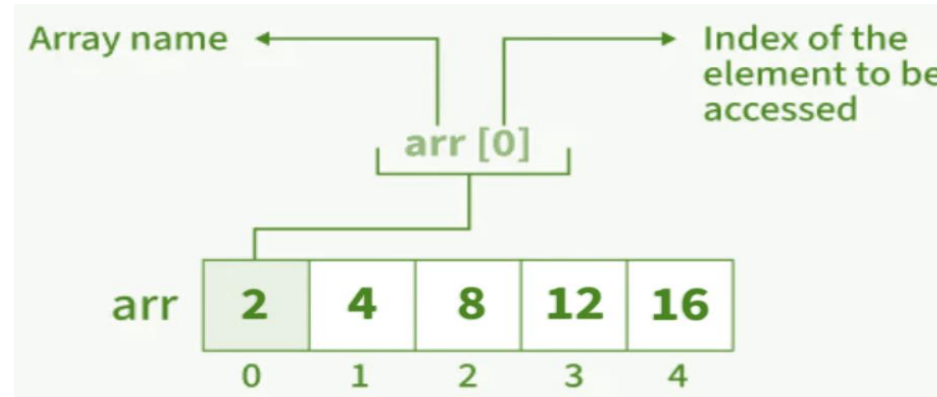


ACCESS THE ELEMENTS OF AN ARRAY

- To access an array element, refer to its **index number**.



- Array indexes start with **0**: `[0]` is the first element. `[1]` is the second element, etc.



EXAMPLE

```
#include <stdio.h>

int main() {

    // array declaration and initialization
    int arr[5] = {2, 4, 8, 12, 16};

    // accessing element at index 2 i.e 3rd element
    printf("%d ", arr[2]);

    // accessing element at index 4 i.e last element
    printf("%d ", arr[4]);

    // accessing element at index 0 i.e first element
    printf("%d ", arr[0]);
    return 0;
}
```

Output

8 16 2

=== Code Execution Successful ===

UPDATE ARRAY ELEMENT

- To change the value of a specific element, refer to the index number:

```
1  #include <stdio.h>
2
3  int main() {
4
5      int arr[5] = {2, 4, 8, 12, 16};
6
7      // Update the first value
8      // of the array
9      arr[0] = 1;
10     printf("%d", arr[0]);
11
12     return 0;
13 }
```

Output

1

=== Code Execution Successful ===

C ARRAY SIZE

- To get the size of an array, you can use the **sizeof** operator:

```
#include <stdio.h>

int main() {

    int myNumbers[] = {10, 25, 50, 75, 100};

    printf("%zu", sizeof(myNumbers));

    return 0;
}
```

Output

20

=== Code Execution Successful ===

Why did the result
show 20 instead of 5,
when the array
contains 5 elements?

- It is because the sizeof operator returns the size of a type in bytes. int type is usually 4 bytes, so from the example above, 4×5 (4 bytes $\times$ 5 elements) = 20

GET THE NUMBER OF ELEMENTS

- If you want to find out how many elements an array has, you can use this formula, which divides the total size of the array by the size of one element

```
#include <stdio.h>

int main() {

    int myNumbers[] = {10, 25, 50, 75};

    int length = sizeof(myNumbers) / sizeof(myNumbers[0]);

    printf("%d", length);

    return 0;
}
```

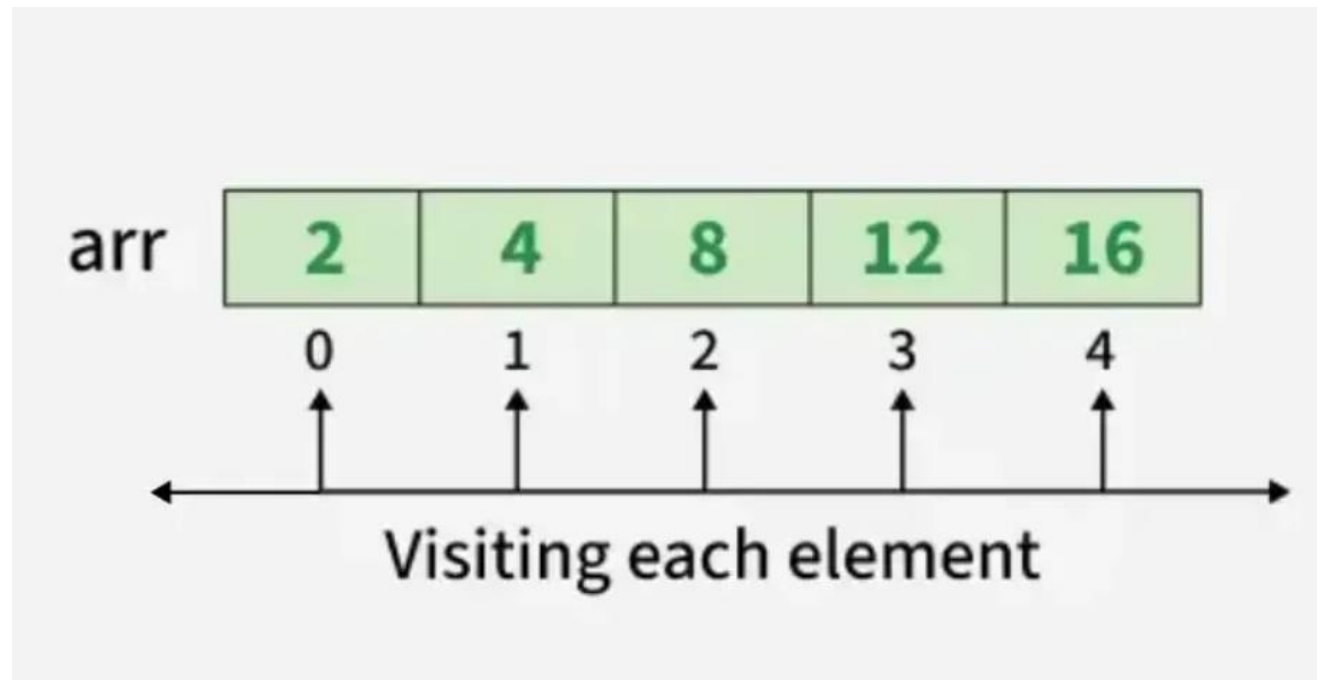
Output

4

=== Code Execution Successful ===

C ARRAY TRAVERSAL

- Array Traversal is the process in which we visit every element of the array in a specific order. For C array traversal, we use loops to iterate through each element of the array.



EXAMPLE

```
int main() {  
    int arr[5] = {2, 4, 8, 12, 16};  
  
    // Print each element of  
    // array using loop  
    printf("Printing Array Elements\n");  
    for(int i = 0; i < 5; i++){  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
  
    // Printing array element in reverse  
    printf("Printing Array Elements in Reverse\n");  
    for(int i = 4; i >= 0; i--){  
        printf("%d ", arr[i]);  
    }  
  
    return 0;  
}
```

```
Printing Array Elements  
2 4 8 12 16  
Printing Array Elements in Reverse  
16 12 8 4 2
```

ARRAY LENGTH IN LOOPS

- using a for loop to go through the elements of an array by writing the size of the array in the loop condition (in previous example the array has 5 elements, so we use $i < 5$). However, this is not ideal, since it will only work for arrays of a specified size:

```
#include <stdio.h>

int main() {
    int myNumbers[] = {2, 4, 8, 12, 16};

    int length = sizeof(myNumbers) / sizeof(myNumbers[0]);
    int i;

    for (i = 0; i < length; i++) {
        printf("%d\n", myNumbers[i]);
    }
    return 0;
}
```

Output

```
2
4
8
12
16
```



STRINGS IN C

STRINGS

- Strings are used for storing text/characters.
- Example :
 "Hello World" is a string of characters.
- Unlike many other programming languages, C does not have a String type.
- use the char type and create an array of characters to make a string in C.
- char greetings[] = "Hello World!";

STRINGS

```
2  #include <stdio.h>
3
4  int main() {
5      char greetings[] = "Hello World!";
6      printf("%s", greetings);
7      return 0;
8  }
```

Output

Hello World!

=== Code Execution Successful ===

you can use the `printf()` function together with the format specifier `%s` to tell C that we are now working with strings

you have to use double quotes (`"`).

ACCESS STRINGS

```
#include <stdio.h>

int main() {
    char greetings[] = "Hello World!";
    //Access String
    printf("%c\n", greetings[0]);
    // Modify String
    greetings[0] = 'J';
    printf("%s", greetings);
    return 0;
}
```

Output

H

Jello World!

=== Code Execution Successful ===

SPECIAL CHARACTERS

- Because strings must be written within quotes, C will misunderstand this string, and generate an error:

Example:

```
char txt[] = "We are the so-called "Vikings" from the north.";
```

- The solution to avoid this problem, is to use the **backslash escape character**.

Escape character	Result	Description
\'	'	Single quote
\"	"	Double quote
\\	\	Backslash

EXAMPLES

```
#include <stdio.h>

int main() {
    char txt1[] = "We are the so-called \"Vikings\" from the north.";
    char txt2[] = "It's alright.";
    char txt3[] = "The character \\ is called backslash.";
    printf("%s\n %s\n %s\n", txt1,txt2,txt3);

    return 0;
}
```

Output

```
We are the so-called "Vikings" from the north.
It's alright.
The character \ is called backslash.
```



Happy Coding!